



REDES E COMUNICAÇÕES 2

SOCKETS

Nuno Fonseca

Sockets

O que são sockets?

- Sockets são mecanismos que permitem a comunicação entre aplicações através da rede. Funcionam como pontos de comunicação entre um cliente e um servidor.
- Uma comunicação por sockets necessita normalmente de:
 - endereço IP
 - porta
 - protocolo de transporte (TCP ou UDP)

Cliente

- inicia a comunicação
- envia pedidos ao servidor

Servidor

- fica à espera de ligações
- processa pedidos
- responde aos clientes

Exemplos comuns:

- browsers e servidores web
- aplicações de chat
- jogos online

TCP (Transmission Control Protocol)

- O artigo explica que o TCP é:
- orientado à ligação
- fiável
- garante entrega e ordem dos dados

Características principais

- confirmação de receção
- retransmissão de pacotes perdidos
- controlo de erros
- comunicação estável
- O TCP estabelece primeiro uma ligação entre cliente e servidor antes da troca de dados.

Exemplos de utilização

- HTTP/HTTPS
- email
- transferência de ficheiros
- bases de dados

UDP (User Datagram Protocol)

- O UDP é apresentado como:
- mais rápido
- sem ligação
- sem garantias de entrega

Características

- menor overhead
- maior velocidade
- não confirma receção
- não garante ordem dos pacotes
- O artigo destaca que o UDP é útil quando velocidade é mais importante que fiabilidade.

Exemplos de utilização

- streaming
- voz sobre IP
- jogos online
- DNS

SOCKETS

TCP	UDP
Fiável	Não fiável
Orientado à ligação	Sem ligação
Mais lento	Mais rápido
Garante ordem	Não garante ordem
Maior controlo	Menor controlo

Portas/Portos

- identificam aplicações dentro do sistema
- permitem múltiplas comunicações simultâneas

Grupo de Portas	Intervalo	Nome Alternativo	Utilização
Well-Known Ports	0 – 1023	System Ports	Serviços e protocolos conhecidos
Registered Ports	1024 – 49151	User Ports	Aplicações e serviços registados
Dynamic / Private Ports	49152 – 65535	Ephemeral Ports	Portas temporárias usadas pelos clientes

Sockets em C#

Namespace

- Em C# deve ser usado o namespace Net e Net.Sockets para ser possível implementar sockets

```
using System.Net;  
using System.Net.Sockets;
```

Classe Socket

A classe Socket é a mais completa e de baixo nível.

Permite:

- TCP
- UDP
- controlo avançado
- comunicação síncrona e assíncrona

```
Socket socket = new Socket(  
    AddressFamily.InterNetwork,  
    SocketType.Stream,  
    ProtocolType.Tcp  
);
```

SOCKETS EM C#

Método	Função
Bind()	Associa IP e porta
Listen()	Coloca socket em escuta
Accept()	Aceita clientes
Connect()	Liga ao servidor
Send()	Envia dados
Receive()	Recebe dados
Close()	Fecha socket

Classe TcpListener

- Classe de alto nível para a criação de servidores com sockets TCP

Método	Função
Start()	Inicia servidor
Stop()	Para servidor
AcceptTcpClient()	Aceita cliente
Pending()	Verifica clientes em espera

Classe TcpListener

```
TcpListener servidor = new TcpListener(  
    IPAddress.Any,  
    5000  
);  
  
servidor.Start();  
  
TcpClient cliente = servidor.AcceptTcpClient();
```

Classe TcpClient

Classe de alto nível para clientes TCP com sockets

Método	Função
Connect()	Liga ao servidor
GetStream()	Obtém NetworkStream
Close()	Fecha ligação

Classe TcpClient

```
TcpClient cliente = new TcpClient();  
  
cliente.Connect("127.0.0.1", 5000);
```

Classe NetworkStream

- Representa o canal de dados TCP

Método	Função
Read()	Recebe dados
Write()	Envia dados
Close()	Fecha stream

```
NetworkStream stream = cliente.GetStream();
```

Classe UdpClient

- Classe de alto nível para sockets UDP

Método	Função
Send()	Envia datagrama
Receive()	Recebe datagrama
Close()	Fecha cliente UDP

```
UdpClient udp = new UdpClient();  
  
udp.Send(dados, dados.Length, "127.0.0.1", 6000);
```

Conversão de dados

- Os dados são transmitidos em bytes (binário)
- É necessário converter os dados de texto para binário e vice-versa

```
byte[] dados = Encoding.UTF8.GetBytes("Olá");
```

```
string texto = Encoding.UTF8.GetString(buffer);
```

Nuno Fonseca

nuno.fonseca@my.istec.pt

